: - Nginx

:

: (Intermediate)

: « Nginx»

: 2026

- : - Nginx
  -
  -
    -
    -
    -
  - 01. Nginx
    - :
    - 1.2.
      - 1.2.1. (Name-Based)
      - 1.2.2. IP- (IP-Based)
      - 1.2.3. (Port-Based)
    - Nginx
    - : Name-Based Virtual Hosts
      - 1.1 —
      - 1.2 — HTML-
      - 1.3 —
      - 1.4 —
      - 1.5 —
    - Windows-
  - 02. SSH
    - : SSH
    - : PowerShell/ OpenSSH
      - 2.1 — (Linux-)
      - 2.2 —
  - 03. DNS
    - : DNS
    - : DNS /etc/hosts
  - 04. HTTP- Python
    - : HTTP
    - : -
      - 4.1 —
      - 4.2 — TCP-
      - 4.3 — HTTP-
      - 4.4 — HTTP-
      - 4.5 — HTTP-
      - 4.6 — (404 Not Found)

- 4.7 —
- 4.8 —
- 4.9 — (access log)
- 4.10 — (403 Forbidden)
- 4.11 — ( )
- 4.12 —

*

– !

```
%%{init: {'theme': 'dark', 'themeVariables': { 'primaryColor': '#87CF3E',
↪  'edgeLabelBackground':'#1e1e1e', 'tertiaryColor': '#2d2d2d'}}}%%
graph TB
    subgraph Windows_Host["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
     ↪  docminer/emoji/1f5a5-fe0f.png}}}}}}$§ Windows 10/11 (Host)"]
        direction TB
        Browser["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f310.png}}}}}}$§ Браузер<br/>http://192.168.56.101"]
        PowerShell["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/1f510.png}}}}}}$§ PowerShell / SSH<br/>ssh student@192.168.56.101"]
        WinSCP["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f4c1.png}}}}}}$§ WinSCP / SFTP"]
    end

    subgraph VBox["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪  1f4e6.png}}}}}}$§ Oracle VirtualBox 7.x"]
        direction TB
        subgraph NetConfig["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪  docminer/emoji/1f310.png}}}}}}$§ Сетевые адаптеры"]
            NAT["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
             ↪  1f30d.png}}}}}}$§ NAT — доступ в Интернет"]
            HostOnly["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
             ↪  emoji/1f50c.png}}}}}}$§ Host-Only Adapter<br/>vboxnet0: 192.168.56.0/24"]
        end

        subgraph VM["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/1f427.png}}}}}}$§ Виртуальная машина<br/>Linux Mint 22 Wilma"]
            direction TB
            subgraph Net["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
             ↪  emoji/1f4e1.png}}}}}}$§ Сетевые интерфейсы"]
                eth0["enp0s3 — NAT<br/>DHCP, выход в Интернет"]
                eth1["enp0s8 — Host-Only<br/>статический: 192.168.56.101"]
            end

            subgraph Services["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
             ↪  docminer/emoji/2699-fe0f.png}}}}}}$§ Услуги"]
```

```
                NginxService["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
                 ↪  docminer/emoji/1f7e2.png}}}}}}$§ Nginx 1.24+<br/>порты 80, 443"]
                SSHService["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
                 ↪  docminer/emoji/1f510.png}}}}}}$§ OpenSSH Server<br/>порт 22 → 2222"]
               DNSLocal["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
                 ↪  emoji/1f4d6.png}}}}}}$§ /etc/hosts<br/>site1.local, site2.local"]
                SSL["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
                 ↪  emoji/1f512.png}}}}}}$§ SSL-сертификаты<br/>self-signed / Let's Encrypt"]
                UFW["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
                 ↪  emoji/1f6e1-fe0f.png}}}}}}$§ UFW Firewall"]
                F2B["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
                 ↪  emoji/1f6ab.png}}}}}}$§ Fail2Ban"]
            end
        end
    end

    Browser -->|"HTTP/HTTPS"| HostOnly
    PowerShell -->|"SSH порт 2222"| HostOnly
    WinSCP -->|"SFTP порт 2222"| HostOnly
    HostOnly --> eth1
    NAT --> eth0
    eth1 --> NginxService
    eth1 --> SSHService
    NginxService --> DNSLocal
    NginxService --> SSL
    eth1 --> UFW
    SSHService --> F2B

    style Windows_Host fill:#0078D4,stroke:#005A9E,color:#fff
    style VM fill:#87CF3E22,stroke:#87CF3E,color:#fff
    style Services fill:#1e1e1e,stroke:#87CF3E,color:#fff
    style Net fill:#2d2d2d,stroke:#555,color:#fff
```

: **!**

Oracle VirtualBox Linux Mint 22 :

Adapter 1	NAT (, Certbot) , DHCP
Adapter 2	Host-Only IP vboxnet0, DHCP Windows-

IP (Linux Mint):

```
# Просмотр доступных интерфейсов
ip addr show

# Редактирование конфигурации netplan
```

```
sudo nano /etc/netplan/00-installer-config.yaml
```

:

```
network:
  version: 2
  ethernets:
    enp0s3:                          # NAT-интерфейс
      dhcp4: true
    enp0s8:                          # Host-Only интерфейс
      dhcp4: false
      addresses:
        - 192.168.56.101/24
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

```
# Применение конфигурации
sudo netplan apply

# Проверка IP-адресов
ip addr show enp0s8
# Ожидаемый результат: inet 192.168.56.101/24
```

Windows:

```
# С ВМ Linux Mint — проверка доступности Windows-хоста
ping -c 4 192.168.56.1
```

> ** :** IP `192.168.56.1` — VirtualBox Windows- . , ,
> .

Linux Mint :

```
#  1. Версия Nginx (должна быть 1.24+)
nginx -v

#  2. Nginx активен
sudo systemctl status nginx --no-pager

#  3. Sudo-привилегии
sudo whoami    # ожидается: root

#  4. Статический IP корректен
ip addr show enp0s8 | grep "192.168.56.101"

#  5. Доступ в Интернет через NAT
ping -c 3 google.com

#  6. SSH-сервер установлен и работает
```

```
sudo systemctl status sshd --no-pager
```

---

01. Nginx

:

: = . 5 5 — .

: Nginx. `Host` HTTP- .

```
%%{init: {'theme': 'dark'}}%%
graph LR
    subgraph Before["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/274c.png}}}}}}$§ Без виртуальных хостов"]
        direction TB
        S1["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f5a5-fe0f.png}}}}}}$§ Сервер 1<br/>192.168.1.10"] --> W1["§$
         ↪ \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f310.png}}}}}}$§ site1.com"]
        S2["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f5a5-fe0f.png}}}}}}$§ Сервер 2<br/>192.168.1.11"] --> W2["§$
         ↪ \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f310.png}}}}}}$§ site2.com"]
        S3["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f5a5-fe0f.png}}}}}}$§ Сервер 3<br/>192.168.1.12"] --> W3["§$
         ↪ \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f310.png}}}}}}$§ site3.com"]
    end

    subgraph After["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/2705.png}}}}}}$§ С виртуальными хостами"]
        direction TB
        Nginx["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 26a1.png}}}}}}$§ Nginx Server<br/>192.168.56.101"] --> VH1["§$
         ↪ \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f4c1.png}}}}}}$§ /var/www/site1.local<br/>server_name site1.local"]
        Nginx --> VH2["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f4c1.png}}}}}}$§ /var/www/site2.local<br/>server_name site2.local"]
        Nginx --> VH3["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f4c1.png}}}}}}$§ /var/www/site3.local<br/>server_name site3.local"]
        VH1 --> D1["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f310.png}}}}}}$§ site1.local"]
        VH2 --> D2["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f310.png}}}}}}$§ site2.local"]
        VH3 --> D3["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f310.png}}}}}}$§ site3.local"]
    end

    style Before fill:#3d1f1f,stroke:#ff4444,color:#fff
    style After fill:#1a3d1a,stroke:#44ff44,color:#fff
```

```
    style Nginx fill:#009639,stroke:#007a2f,color:#fff
```

1.2.

Nginx :

```
%%{init: {'theme': 'dark'}}%%
graph TB
    subgraph Methods["Методы виртуального хостинга"]
        direction LR

        subgraph NameBased["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪ docminer/emoji/1f7e2.png}}}}}$§ Name-Based<br/>(рекомендуется)"]
            NB["Один IP → много сайтов<br/>Разделение по Host-заголовку<br/>listen 80;<br/>server_name
             ↪ site1.local;"]
        end

        subgraph IPBased["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪ docminer/emoji/1f7e1.png}}}}}$§ IP-Based"]
            IP["Каждому сайту — свой IP<br/>listen 192.168.56.101:80;<br/>listen 192.168.56.102:80;"]
        end

        subgraph PortBased["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪ docminer/emoji/1f7e0.png}}}}}$§ Port-Based"]
            PB["Разные порты для разных сайтов<br/>listen 80;<br/>listen 8080;<br/>listen 3000;"]
        end
    end

    style Methods fill:#1e1e1e,stroke:#555,color:#fff
    style NameBased fill:#1a3d1a,stroke:#44ff44,color:#fff
    style IPBased fill:#3d3d1a,stroke:#ffff44,color:#fff
    style PortBased fill:#3d281a,stroke:#ffaa44,color:#fff
```

1.2.1. (Name-Based) .

`Host` HTTP/1.1 .

```
server {
    listen 80;
    server_name site1.local;
    root /var/www/site1.local;
}

server {
    listen 80;
    server_name site2.local;
    root /var/www/site2.local;
}
```

IP- HTTP/1.1

Table 2 – continued

IP- Host ( )
–

1.2.2. IP- (IP-Based) - IP- .

IP- .

```
server {
    listen 192.168.1.10:80;
    root /var/www/site1.local;
}

server {
    listen 192.168.1.11:80;
    root /var/www/site2.local;
}
```

IP-
HTTP IPv4-

1.2.3. (Port-Based) - TCP- .

```
server {
    listen 80;
    root /var/www/site1.local;
}

server {
    listen 8080;
    root /var/www/site2.local;
}
```

URL ( )

Nginx

Nginx :

```
/etc/nginx/
├── nginx.conf              # Главный конфигурационный файл
├── sites-available/        # Хранилище всех конфигурационных блоков серверов
│   ├── site1.local         # Конфигурация первого виртуального хоста
│   ├── site2.local         # Конфигурация второго виртуального хоста
```

```
│   └── default             # Конфигурация по умолчанию
└── sites-enabled/          # Активные конфигурации (символические ссылки)
    ├── site1.local -> ../sites-available/site1.local
    └── site2.local -> ../sites-available/site2.local
```

> : (symlinks) **sites-available/** **sites-enabled/** Debian/Ubuntu.
,
.

: **Name-Based Virtual Hosts**

🎯 : site1.local site2.local IP 192.168.56.101, Windows- .

1.1 —

```
# Создаём корневые директории сайтов
sudo mkdir -p /var/www/site1.local /var/www/site2.local

# Назначаем права (замените student на вашего пользователя)
sudo chown -R $USER:$USER /var/www/site1.local /var/www/site2.local

# Устанавливаем права доступа
sudo chmod -R 755 /var/www/site1.local /var/www/site2.local

# Проверяем
ls -la /var/www/
```

1.2 — HTML-

```
# === Сайт 1 ===
cat > /var/www/site1.local/index.html << 'EOF'
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Сайт 1 — Виртуальный хост Nginx</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
            color: #eaeaea;
            min-height: 100vh;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            text-align: center;
        }
```

```
        .container { max-width: 600px; padding: 40px; }
        h1 { font-size: 2.5em; margin-bottom: 20px; color: #e94560; }
        .badge {
            display: inline-block;
            background: #0f3460;
            padding: 10px 20px;
            border-radius: 25px;
            margin: 10px;
            font-size: 0.9em;
        }
        .success { color: #4ecca3; font-size: 1.2em; margin-top: 30px; }
        .ip { color: #f5a623; margin-top: 15px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f389.png}}}}}}$§ Сайт 1 работает!</h1>
        <span class="badge">Nginx</span>
        <span class="badge">Virtual Host</span>
        <span class="badge">Linux Mint 22</span>
        <p class="success">§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪  docminer/emoji/2705.png}}}}}}$§ Первый виртуальный хост успешно сконфигурирован</p>
        <p class="ip">Сервер: 192.168.56.101 | Host-Only Adapter</p>
    </div>
</body>
</html>
EOF

# === Сайт 2 ===
cat > /var/www/site2.local/index.html << 'EOF'
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Сайт 2 — Виртуальный хост Nginx</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #0f2027 0%, #203a43 50%, #2c5364 100%);
            color: #eaeaea;
            min-height: 100vh;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            text-align: center;
        }
        .container { max-width: 600px; padding: 40px; }
        h1 { font-size: 2.5em; margin-bottom: 20px; color: #4ecca3; }
```

```
        .badge {
            display: inline-block;
            background: #1a3c40;
            padding: 10px 20px;
            border-radius: 25px;
            margin: 10px;
            font-size: 0.9em;
        }
        .success { color: #f5a623; font-size: 1.2em; margin-top: 30px; }
        .ip { color: #e94560; margin-top: 15px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
        ↪  1f680.png}}}}}}$§ Сайт 2 работает!</h1>
        <span class="badge">Nginx</span>
        <span class="badge">Virtual Host</span>
        <span class="badge">Linux Mint 22</span>
        <p class="success">§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
        ↪  docminer/emoji/2705.png}}}}}}$§ Второй виртуальный хост успешно сконфигурирован</p>
        <p class="ip">Сервер: 192.168.56.101 | Host-Only Adapter</p>
    </div>
</body>
</html>
EOF
```

1.3 —

```
# Создаём конфиг для site1.local
sudo nano /etc/nginx/sites-available/site1.local
```

:

```
server {
    listen 80;
    listen [::]:80;

    server_name site1.local;
    root /var/www/site1.local;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    # Логи для отладки
    access_log /var/log/nginx/site1.access.log;
    error_log /var/log/nginx/site1.error.log;
}
```

```
# Создаём конфиг для site2.local
```

```
sudo nano /etc/nginx/sites-available/site2.local
```

```
server {
    listen 80;
    listen [::]:80;

    server_name site2.local;
    root /var/www/site2.local;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/site2.access.log;
    error_log /var/log/nginx/site2.error.log;
}
```

1.4 —

```
# Активируем через символические ссылки
sudo ln -s /etc/nginx/sites-available/site1.local /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/site2.local /etc/nginx/sites-enabled/

# Удаляем дефолтный хост (чтобы не конфликтовал)
sudo rm -f /etc/nginx/sites-enabled/default

# Проверка синтаксиса — ВСЕГДА перед reload!
sudo nginx -t

# Перезагрузка конфигурации
sudo systemctl reload nginx

# Проверка статуса
sudo systemctl status nginx --no-pager
```

1.5 —

```
# Редактируем файл hosts
sudo nano /etc/hosts
```

:

```
192.168.56.101    site1.local
192.168.56.101    site2.local
```

:

```
curl -s http://site1.local | grep "title"
curl -s http://site2.local | grep "title"
```

Windows-

🪟 **Windows** **`hosts` Windows!**

hosts **Windows:** `C:\Windows\System32\drivers\etc\hosts`

→ → → .

:

```
192.168.56.101    site1.local
192.168.56.101    site2.local
```

(`Ctrl+S`).

Windows:

```
# В PowerShell или CMD:
ping site1.local
ping site2.local
```

Windows:

- 🌐 `http://site1.local` — 1 ( )
- 🌐 `http://site2.local` — 2 ( )

```
%%{init: {'theme': 'dark'}}%%
sequenceDiagram
    participant W as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/1f5a5-fe0f.png}}}}}}$§ Windows Host
    participant B as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/1f310.png}}}}}}$§ Браузер
    participant H as hosts file
    participant V as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/1f4e6.png}}}}}}$§ VirtualBox<br/>Host-Only
    participant N as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪ emoji/26a1.png}}}}}}$§ Nginx<br/>192.168.56.101

    W->>B: Открыть http://site1.local
    B->>H: Резолв site1.local
    H-->>B: 192.168.56.101
    B->>V: HTTP GET / → 192.168.56.101:80
    V->>N: Проброс пакета
    N->>N: Host: site1.local → server_name
    N->>V: HTTP 200 OK + HTML
    V->>B: Ответ
    B-->>W: Отображение Сайта 1 §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
     ↪ docminer/emoji/1f389.png}}}}}}$§
```

✅ **:** Windows `http://site1.local` `http://site2.local`.

---

02. SSH

: SSH

```
%%{init: {'theme': 'dark'}}%%
graph LR
    subgraph Insecure["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/274c.png}}}}}}$§ Небезопасные протоколы"]
        Telnet["Telnet"] -->|"Передача в открытом виде"| Sniffer["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f50d.png}}}}}}$§ Перехват пароля"]
        FTP["FTP"] -->|"Передача в открытом виде"| Sniffer
    end

    subgraph Secure["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/2705.png}}}}}}$§ SSH (Secure Shell)"]
        SSH["SSH"] -->|"§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/1f510.png}}}}}}$§ Шифрование "| Tunnel["Зашифрованный туннель"]
        Tunnel --> Safe["Безопасная передача данных"]
    end

    style Insecure fill:#3d1f1f,stroke:#ff4444,color:#fff
    style Secure fill:#1a3d1a,stroke:#44ff44,color:#fff
    style Sniffer fill:#5d1f1f,stroke:#ff0000,color:#fff
```

:

```
%%{init: {'theme': 'dark'}}%%
sequenceDiagram
    autonumber
    participant C as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f4bb.png}}}}}}$§ Клиент<br/>(PowerShell/ssh)
    participant Pr as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f512.png}}}}}}$§ Приватный ключ<br/>~/.ssh/id_ed25519
    participant S as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f5a5-fe0f.png}}}}}}$§ Сервер<br/>192.168.56.101
    participant Pu as §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f513.png}}}}}}$§ Публичный ключ<br/>~/.ssh/authorized_keys

    C->>S: Подключение: ssh student@192.168.56.101
    S->>S: Генерация случайного challenge
    S-->>C: Отправка challenge
    C->>Pr: Подписать challenge
    Pr-->>C: Подпись
    C->>S: Отправка подписи
    S->>Pu: Проверка подписи<br/>публичным ключом
    Pu-->>S: §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪  2705.png}}}}}}$§ Подпись валидна
    S-->>C: §$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪  1f7e2.png}}}}}}$§ Доступ разрешён!
```

:

Ed25519	256	— ,
RSA	4096	
ECDSA	521	NSA-

: PowerShell/ OpenSSH

2.1 — (Linux-)

```
# Создаём директорию .ssh (если нет)
mkdir -p ~/.ssh && chmod 700 ~/.ssh

# Генерация Ed25519 ключа
ssh-keygen -t ed25519 -a 100 -C "student@lab-mint"

# При запросе:
# Enter file: [Enter — по умолчанию ~/.ssh/id_ed25519]
# Enter passphrase: [Введите надёжную passphrase — ОБЯЗАТЕЛЬНО!]
# Confirm passphrase: [Повторите]
```

:

```
ls -la ~/.ssh/
# Должно быть:
# - id_ed25519      (приватный, права 600)
# - id_ed25519.pub  (публичный, права 644)
```

2.2 —

```
# Копируем публичный ключ в authorized_keys
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Проверяем содержимое
cat ~/.ssh/authorized_keys
# Должна быть строка, начинающаяся с ssh-ed25519 ...
```

---

03. DNS

: DNS

```
%%{init: {'theme': 'dark'}}%%
graph TD
    subgraph DNSQuery["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
    ↪ emoji/1f50d.png}}}}}}$§ Рекурсивное разрешение DNS"]
        direction TB
```

```
        User["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f464.png}}}}}}$§ Пользователь<br/>вводит site1.local"] --> CheckCache{"§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f4cb.png}}}}}}$§ Проверка<br/>кэша /etc/hosts"}

        CheckCache -->|"§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/2705.png}}}}}}$§ Найдено"| Direct["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f3af.png}}}}}}$§ Прямое<br/>соединение<br/>192.168.56.101"]
        CheckCache -->|"§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/274c.png}}}}}}$§ Не найдено"| Resolver["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f310.png}}}}}}$§ DNS-резолвер<br/>8.8.8.8"]

        Resolver --> Root["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪  docminer/emoji/1f4e1.png}}}}}}$§ Корневой сервер<br/>."]
        Root --> TLD["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/1f4e1.png}}}}}}$§ TLD-сервер<br/>.local / .com / .ru"]
        TLD --> AuthNS["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪  emoji/1f4e1.png}}}}}}$§ Авторитарный NS<br/>ns1.example.com"]
        AuthNS --> Answer["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪  docminer/emoji/1f4c4.png}}}}}}$§ DNS-ответ<br/>A-запись"]
        Answer --> Connect["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
         ↪  docminer/emoji/1f517.png}}}}}}$§ TCP-соединение<br/>с 192.168.56.101"]
    end

    style Direct fill:#1a3d1a,stroke:#44ff44,color:#fff
    style CheckCache fill:#3d3d1a,stroke:#ffff44,color:#fff
    style Connect fill:#1a3d1a,stroke:#44ff44,color:#fff
```

DNS- :

A	→ IPv4-	`site1.local. IN A 192.168.56.101`
AAAA	→ IPv6-	`site1.local. IN AAAA ::1`
CNAME	( )	`www.site1.local. IN CNAME site1.local.`
MX		`site1.local. IN MX 10 mail.site1.local.`

: DNS /etc/hosts

DNS- hosts.

```
%%{init: {'theme': 'dark'}}%%
graph LR
    subgraph VMHosts["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f427.png}}}}}}$§ /etc/hosts на ВМ Linux Mint"]
```

```
        V1["192.168.56.101 site1.local"] --> NginxVM["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  26a1.png}}}}}}$§ Nginx<br/>раздаёт site1"]
        V2["192.168.56.101 site2.local"] --> NginxVM2["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  26a1.png}}}}}}$§ Nginx<br/>раздаёт site2"]
    end

    subgraph WinHosts["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
     ↪  emoji/1f5a5-fe0f.png}}}}}}$§ C:\Windows\System32\drivers\etc\hosts"]
        W1["192.168.56.101 site1.local"] --> Browser1["§$
         ↪  \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪  1f310.png}}}}}}$§ Браузер<br/>Windows"]
        W2["192.168.56.101 site2.local"] --> Browser1
    end

    Browser1 -->|"HTTP запрос"| NginxVM
    Browser1 -->|"HTTP запрос"| NginxVM2

    style VMHosts fill:#1a3d3d,stroke:#44ffff,color:#fff
    style WinHosts fill:#1a1a3d,stroke:#4444ff,color:#fff
```

Linux Mint:

```
# Проверка текущего содержимого
cat /etc/hosts

# Добавление записей (если ещё не добавлены в Модуле 01)
sudo tee -a /etc/hosts << 'EOF'
192.168.56.101    site1.local
192.168.56.101    site2.local
EOF

# Проверка
cat /etc/hosts | grep site
```

Windows- :

```
# Открыть Блокнот от имени Администратора → C:\Windows\System32\drivers\etc\hosts
# Добавить:
# 192.168.56.101    site1.local
# 192.168.56.101    site2.local

# Проверка в PowerShell:
ping site1.local
# Ожидается ответ от 192.168.56.101
```

DNS:

```
# В Linux:
getent hosts site1.local
nslookup site1.local
dig site1.local
```

```
# Трассировка полного пути
dig +trace google.com
```

📚 : `/etc/hosts` (DNS). `/etc/nsswitch.conf` (`hosts: files dns`). DNS- .

---

04. HTTP- Python

🎯 : HTTP, - TCP- Python, HTTP- , , .

🔗 : github.com/fa-python-network/6_Web_server

: HTTP

HTTP (HyperText Transfer Protocol) — , . HTTP « — » ( ) .

HTTP-

```
%%{init: {'theme': 'dark'}}%%
graph TB
    subgraph HTTPRequest["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4e8.png}}}}}}$§ HTTP-запрос клиента"]
        direction TB
        RequestLine["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4cd.png}}}}}}$§ Стартовая строка<br/>GET /index.html HTTP/1.1<br/>↑метод ↑ресурс ↑версия"]
        Headers["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4cb.png}}}}}}$§ Заголовки<br/>Host: localhost:8080<br/>User-Agent: Mozilla/5.0<br/>Accept: text/html<br/>Connection: close"]
        EmptyLine["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2b1c.png}}}}}}$§ Пустая строка<br/>(обязательный разделитель)"]
        Body["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4c4.png}}}}}}$§ Тело запроса<br/>(для POST/PUT; при GET — пустое)"]
    end

    RequestLine --> Headers
    Headers --> EmptyLine
    EmptyLine --> Body

    style RequestLine fill:#1a3d1a,stroke:#44ff44,color:#fff
    style Headers fill:#3d3d1a,stroke:#ffff44,color:#fff
    style EmptyLine fill:#2d2d2d,stroke:#555,color:#fff
    style Body fill:#1a1a3d,stroke:#4444ff,color:#fff
```

(Request Line) — . , :

,	GET, POST, HEAD, PUT, DELETE
URI	/, /index.html, /css/style.css
HTTP	HTTP/1.0, HTTP/1.1

> 💡 **:** GET, - . GET
> — .

HTTP-

```
%%{init: {'theme': 'dark'}}%%
graph TB
    subgraph HTTPResponse["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
     ↪ docminer/emoji/1f4e4.png}}}}}}$§ HTTP-ответ сервера"]
        direction TB
        StatusLine["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f4cd.png}}}}}}$§ Статусная строка<br/>HTTP/1.1 200 OK<br/>↑версия ↑код ↑пояснение"]
        RespHeaders["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/1f4cb.png}}}}}}$§ Заголовки ответа<br/>Date: Mon, 23 Apr 2026...<br/>Content-Type:
         ↪ text/html<br/>Content-Length: 1234<br/>Server: MyPythonServer/1.0"]
        RespEmptyLine["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
         ↪ emoji/2b1c.png}}}}}}$§ Пустая строка<br/>(обязательный разделитель)"]
        RespBody["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
         ↪ 1f4c4.png}}}}}}$§ Тело ответа<br/>(содержимое HTML-файла)"]
    end

    StatusLine --> RespHeaders
    RespHeaders --> RespEmptyLine
    RespEmptyLine --> RespBody

    style StatusLine fill:#1a3d1a,stroke:#44ff44,color:#fff
    style RespHeaders fill:#3d3d1a,stroke:#ffff44,color:#fff
    style RespEmptyLine fill:#2d2d2d,stroke:#555,color:#fff
    style RespBody fill:#1a1a3d,stroke:#4444ff,color:#fff
```

HTTP

200	OK	
301	Moved Permanently	URL
400	Bad Request	
403	Forbidden	
404	Not Found	
500	Internal Server Error	
505	HTTP Version Not Supported	HTTP

Python- **socket** Python socket. API TCP/IP.

```
%%{init: {'theme': 'dark'}}%%
graph LR
    subgraph SocketFlow["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
    ↪ emoji/1f50c.png}}}}}}$§ Жизненный цикл TCP-сервера"]
        direction TB
        S1["socket()<br/>Создание сокета"] --> S2["bind()<br/>Привязка к адресу:порту"]
        S2 --> S3["listen()<br/>Перевод в режим ожидания"]
        S3 --> S4["accept()<br/>Принятие соединения"]
        S4 --> S5["recv() / send()<br/>Обмен данными"]
        S5 --> S6["close()<br/>Закрытие соединения"]
        S5 -->|"Следующий клиент"| S4
    end

    style S1 fill:#1a3d1a,stroke:#44ff44,color:#fff
    style S2 fill:#1a3d1a,stroke:#44ff44,color:#fff
    style S3 fill:#3d3d1a,stroke:#ffff44,color:#fff
    style S4 fill:#3d3d1a,stroke:#ffff44,color:#fff
    style S5 fill:#1a1a3d,stroke:#4444ff,color:#fff
    style S6 fill:#3d1f1f,stroke:#ff4444,color:#fff
```

⚠️ **:** 80 — (< 1024). (sudo). **8080**, sudo. URL : `http://localhost:8080`.

: -

🎓 **:** . (`server_v1.py`, `server_v2.py` ..), .

4.1 — HTML- .

```
# Создаём рабочую директорию в домашней папке
mkdir -p ~/pywebserver/www
cd ~/pywebserver

# Создаём простую тестовую страницу 1.html
cat > ~/pywebserver/www/1.html << 'EOF'
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Первый файл</title>
    <style>
        body { font-family: Arial, sans-serif; background: #1a1a2e; color: #fff;
               display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }
        h1 { color: #4ecca3; }
    </style>
</head>
<body>
    <h1>Первый файл</h1>
</body>
```

```
</html>
EOF

# Создаём простую тестовую страницу 2.html
cat > ~/pywebserver/www/2.html << 'EOF'
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Второй файл</title>
    <style>
        body { font-family: Arial, sans-serif; background: #16213e; color: #fff;
               display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }
        h1 { color: #e94560; }
    </style>
</head>
<body>
    <h1>Второй файл</h1>
</body>
</html>
EOF

# Создаём главную страницу index.html
cat > ~/pywebserver/www/index.html << 'EOF'
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Главная страница — Python Web Server</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; background: linear-gradient(135deg, #1a1a2e 0%,
         ↪  #16213e 100%);
               color: #eaeaea; min-height: 100vh; display: flex; flex-direction: column;
               align-items: center; justify-content: center; text-align: center; margin: 0; }
        h1 { color: #4ecca3; font-size: 2.5em; margin-bottom: 10px; }
        .badge { background: #0f3460; padding: 8px 16px; border-radius: 20px; margin: 5px; display:
         ↪  inline-block; }
        .info { color: #f5a623; margin-top: 20px; }
        a { color: #4ecca3; }
    </style>
</head>
<body>
    <h1>Python Web Server работает!</h1>
    <span class="badge">Python 3</span>
    <span class="badge">socket</span>
    <span class="badge">HTTP/1.1</span>
    <p class="info">Тестовые страницы: <a href="1.html">1.html</a> | <a href="2.html">2.html</a></p>
</body>
</html>
EOF
```

```
# Проверяем структуру
ls -la ~/pywebserver/www/
```

✅ : ~/pywebserver/www/ : index.html, 1.html, 2.html.

4.2 — TCP- TCP- , ,
.

```
# Создаём файл сервера
nano ~/pywebserver/server.py
```

:

```
import socket

# Создаём TCP-сокет
# socket.AF_INET — адресное семейство IPv4
# socket.SOCK_STREAM — протокол TCP (потоковый)
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    # Пытаемся привязаться к порту 80 (стандартный HTTP-порт)
    # '' означает привязку ко всем доступным сетевым интерфейсам
    sock.bind(('', 80))
    print("Using port 80")
except OSError:
    # Если порт 80 занят (требует root) — используем 8080
    sock.bind(('', 8080))
    print("Using port 8080")

# Переводим сокет в режим прослушивания
# Параметр 5 — размер очереди ожидающих соединений (backlog)
sock.listen(5)

print("Сервер запущен. Ожидание подключений...")

# accept() блокирует выполнение до подключения клиента
# Возвращает кортеж: (новый_сокет_для_общения, адрес_клиента)
conn, addr = sock.accept()
print("Connected", addr)

# Получаем данные от клиента (максимум 8192 байт)
data = conn.recv(8192)
msg = data.decode()

# Выводим HTTP-запрос на экран
print(msg)

# Закрываем соединение
conn.close()
```

:

```
# Запускаем сервер в терминале 1
cd ~/pywebserver
python3 server.py
# Ожидаемый вывод:
# Using port 8080
# Сервер запущен. Ожидание подключений...
```

```
# В терминале 2 (не закрывайте первый!) отправляем тестовый запрос:
curl http://localhost:8080

# Или откройте браузер и перейдите на http://localhost:8080
```

🔍 : HTTP- . :

```
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/7.81.0
Accept: */*
```

⛔ : . , .
.

4.3 — HTTP- HTTP- . :
.

```
nano ~/pywebserver/server.py
```

:

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    sock.bind(('', 80))
    print("Using port 80")
except OSError:
    sock.bind(('', 8080))
    print("Using port 8080")

sock.listen(5)
print("Сервер запущен. Ожидание подключений...")

conn, addr = sock.accept()
print("Connected", addr)

data = conn.recv(8192)
msg = data.decode()
print(msg)
```

```
# Формируем HTTP-ответ
# ВАЖНО: пустая строка между заголовками и телом — ОБЯЗАТЕЛЬНА!
resp = """HTTP/1.1 200 OK
Server: SelfMadeServer v0.0.1
Content-type: text/html
Connection: close

Hello, webworld!"""

# Отправляем ответ клиенту (кодируем строку в байты)
conn.send(resp.encode())

# Закрываем соединение
conn.close()
```

```
# Перезапускаем сервер
cd ~/pywebserver
python3 server.py
```

```
# В другом терминале проверяем
curl -v http://localhost:8080
# Ожидается: Hello, webworld!
```

💡 **:** `-v` curl — , . HTTP.

📚 **:** `resp.` (`\n`) . . HTTP- .

4.4 — HTTP- :

1. HTTP-
2.
3.
4. (`/`) — `index.html`

```
import socket
import os

# === Конфигурация ===
WWW_DIR = os.path.join(os.path.dirname(__file__), 'www')
DEFAULT_PAGE = 'index.html'

def parse_request(request_text):
    """Извлекает путь к ресурсу из первой строки HTTP-запроса.

    Пример: 'GET /1.html HTTP/1.1' -> '/1.html'
    """
    lines = request_text.splitlines()
```

```python
    if not lines:
        return None

    first_line = lines[0]
    parts = first_line.split()

    # parts[0] = метод (GET), parts[1] = путь, parts[2] = версия HTTP
    if len(parts) < 3:
        return None

    return parts[1]

def get_file_path(uri):
    """Преобразует URI в безопасный путь к файлу.

    /        -> www/index.html
    /1.html  -> www/1.html
    """
    # Убираем начальный слеш
    if uri.startswith('/'):
        uri = uri[1:]

    # Если путь пустой — используем страницу по умолчанию
    if not uri:
        uri = DEFAULT_PAGE

    # Формируем полный путь
    return os.path.join(WWW_DIR, uri)

# === Основной код сервера ===
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    sock.bind(('', 80))
    print("Using port 80")
except OSError:
    sock.bind(('', 8080))
    print("Using port 8080")

sock.listen(5)
print("Сервер запущен. Ожидание подключений...")

while True:
    conn, addr = sock.accept()
    print("Connected", addr)

    data = conn.recv(8192)
    msg = data.decode()
    print(msg)

    # Парсим запрос
```

Generated by Kimi.ai

```
    uri = parse_request(msg)
    print(f"Запрошен ресурс: {uri}")

    # Определяем путь к файлу
    file_path = get_file_path(uri)
    print(f"Путь к файлу: {file_path}")

    # Проверяем существование файла
    if os.path.exists(file_path) and os.path.isfile(file_path):
        # Читаем содержимое файла
        with open(file_path, 'r') as f:
            content = f.read()

        # Формируем успешный ответ
        resp = f"""HTTP/1.1 200 OK
Content-type: text/html
Connection: close

{content}"""
    else:
        # Файл не найден
        resp = """HTTP/1.1 404 Not Found
Content-type: text/html
Connection: close

<h1>404 Not Found</h1>"""

    conn.send(resp.encode())
    conn.close()
```

```
# Перезапускаем и тестируем
cd ~/pywebserver
python3 server.py
```

```
# Тесты в другом терминале:
curl http://localhost:8080          # -> содержимое index.html
curl http://localhost:8080/1.html   # -> содержимое 1.html
curl http://localhost:8080/2.html   # -> содержимое 2.html
curl http://localhost:8080/none     # -> 404 Not Found
```

⚠️ **:** while True: — , .
: , .

4.5 — **HTTP-** - .
.

Date		GMT	Date: Mon, 23 Apr 2026 12:00:00 GMT
Content-Type	MIME-		text/html, text/css, image/png

Table 9 – continued

Server	MyPythonServer/1.0
Content-Length	Content-Length: 1234
Connection	close keep-alive

```python
import socket
import os
import datetime

WWW_DIR = os.path.join(os.path.dirname(__file__), 'www')
DEFAULT_PAGE = 'index.html'

def parse_request(request_text):
    lines = request_text.splitlines()
    if not lines:
        return None
    parts = lines[0].split()
    return parts[1] if len(parts) >= 3 else None

def get_file_path(uri):
    if uri.startswith('/'):
        uri = uri[1:]
    if not uri:
        uri = DEFAULT_PAGE
    return os.path.join(WWW_DIR, uri)

def build_headers(content_length, content_type='text/html', connection='close'):
    """Формирует обязательные HTTP-заголовки."""
    # GMT-дата в формате RFC 7231
    date_str = datetime.datetime.utcnow().strftime('%a, %d %b %Y %H:%M:%S GMT')

    headers = f"""Date: {date_str}
Server: MyPythonServer/1.0
Content-Type: {content_type}
Content-Length: {content_length}
Connection: {connection}"""
    return headers

# === Сервер ===
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    sock.bind(('', 80))
    print("Using port 80")
except OSError:
    sock.bind(('', 8080))
    print("Using port 8080")

sock.listen(5)
```

```
print("Сервер запущен. Ожидание подключений...")

while True:
    conn, addr = sock.accept()
    print("Connected", addr)

    data = conn.recv(8192)
    msg = data.decode()

    uri = parse_request(msg)
    file_path = get_file_path(uri)

    if os.path.exists(file_path) and os.path.isfile(file_path):
        with open(file_path, 'r') as f:
            content = f.read()

        headers = build_headers(len(content.encode('utf-8')))
        resp = f"""HTTP/1.1 200 OK
{headers}

{content}"""
    else:
        body = "<h1>404 Not Found</h1>"
        headers = build_headers(len(body.encode('utf-8')))
        resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""

    conn.send(resp.encode())
    conn.close()
```

💡 : Content-Length , . (UTF-8) len(content.encode('utf-8')), len(content).

4.6 — (404 Not Found) 404, , .

```
def generate_error_page(status_code, status_text, description=""):
    """Генерирует HTML-страницу для ошибки."""
    html = f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>{status_code} {status_text}</title>
    <style>
        body {{ font-family: 'Segoe UI', sans-serif; background: #1a1a2e; color: #fff;
                display: flex; flex-direction: column; align-items: center;
                justify-content: center; height: 100vh; margin: 0; text-align: center; }}
        .code {{ font-size: 6em; color: #e94560; margin: 0; }}
```

```
        .text {{ font-size: 1.5em; color: #f5a623; }}
        .desc {{ color: #888; margin-top: 20px; }}
    </style>
</head>
<body>
    <p class="code">{status_code}</p>
    <p class="text">{status_text}</p>
    <p class="desc">{description}</p>
</body>
</html>"""
    return html

# В основном цикле замените блок 404 на:
else:
    body = generate_error_page(404, "Not Found", f"Файл не найден: {uri}")
    headers = build_headers(len(body.encode('utf-8')))
    resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""
```

✅ : http://localhost:8080/nonexistent — 404.

4.7 — . threading.

```
import socket
import os
import datetime
import threading  # Импорт модуля для многопоточности

WWW_DIR = os.path.join(os.path.dirname(__file__), 'www')
DEFAULT_PAGE = 'index.html'

def parse_request(request_text):
    lines = request_text.splitlines()
    if not lines:
        return None
    parts = lines[0].split()
    return parts[1] if len(parts) >= 3 else None

def get_file_path(uri):
    if uri.startswith('/'):
        uri = uri[1:]
    if not uri:
        uri = DEFAULT_PAGE
    return os.path.join(WWW_DIR, uri)

def build_headers(content_length, content_type='text/html', connection='close'):
    date_str = datetime.datetime.utcnow().strftime('%a, %d %b %Y %H:%M:%S GMT')
    return f"""Date: {date_str}
```

```
Server: MyPythonServer/1.0
Content-Type: {content_type}
Content-Length: {content_length}
Connection: {connection}"""

def generate_error_page(status_code, status_text, description=""):
    html = f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>{status_code} {status_text}</title>
    <style>
        body {{ font-family: 'Segoe UI', sans-serif; background: #1a1a2e; color: #fff;
                display: flex; flex-direction: column; align-items: center;
                justify-content: center; height: 100vh; margin: 0; text-align: center; }}
        .code {{ font-size: 6em; color: #e94560; margin: 0; }}
        .text {{ font-size: 1.5em; color: #f5a623; }}
        .desc {{ color: #888; margin-top: 20px; }}
    </style>
</head>
<body>
    <p class="code">{status_code}</p>
    <p class="text">{status_text}</p>
    <p class="desc">{description}</p>
</body>
</html>"""
    return html

def handle_client(conn, addr):
    """Обрабатывает одно клиентское соединение в отдельном потоке."""
    print(f"[+] Клиент {addr} подключён")

    try:
        data = conn.recv(8192)
        if not data:
            return

        msg = data.decode()
        uri = parse_request(msg)

        if uri is None:
            # Некорректный запрос
            body = generate_error_page(400, "Bad Request", "Некорректный формат HTTP-запроса")
            headers = build_headers(len(body.encode('utf-8')))
            resp = f"""HTTP/1.1 400 Bad Request
{headers}

{body}"""
        else:
            file_path = get_file_path(uri)
```

```
            if os.path.exists(file_path) and os.path.isfile(file_path):
                with open(file_path, 'r') as f:
                    content = f.read()
                headers = build_headers(len(content.encode('utf-8')))
                resp = f"""HTTP/1.1 200 OK
{headers}

{content}"""
            else:
                body = generate_error_page(404, "Not Found", f"Файл не найден: {uri}")
                headers = build_headers(len(body.encode('utf-8')))
                resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""

        conn.send(resp.encode())
    except Exception as e:
        print(f"[!] Ошибка обработки клиента {addr}: {e}")
    finally:
        conn.close()
        print(f"[-] Клиент {addr} отключён")

# === Основной цикл сервера ===
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # Повторное использование адреса

try:
    sock.bind(('', 80))
    print("Using port 80")
except OSError:
    sock.bind(('', 8080))
    print("Using port 8080")

sock.listen(5)
print("Многопоточный сервер запущен. Ожидание подключений...")
print("Нажмите Ctrl+C для остановки.")

try:
    while True:
        conn, addr = sock.accept()
        # Создаём новый поток для каждого клиента
        client_thread = threading.Thread(target=handle_client, args=(conn, addr))
        client_thread.daemon = True  # Поток завершится при выходе из программы
        client_thread.start()
        print(f"[i] Активных потоков: {threading.active_count() - 1}")
except KeyboardInterrupt:
    print("\n[!] Сервер остановлен по запросу пользователя")
finally:
    sock.close()
```

💡 : socket.SO_REUSEADDR .
Address already in use.

🧪 : .
.

4.8 — config.ini. .

```
# Создаём файл конфигурации
cat > ~/pywebserver/config.ini << 'EOF'
[server]
port = 8080
www_dir = www
max_request_size = 8192

[headers]
server_name = MyPythonServer/1.0
EOF
```

```
import socket
import os
import datetime
import threading
import configparser  # Модуль для работы с INI-файлами

# === Загрузка конфигурации ===
config = configparser.ConfigParser()
config.read('config.ini')

PORT = config.getint('server', 'port', fallback=8080)
WWW_DIR = config.get('server', 'www_dir', fallback='www')
MAX_REQUEST_SIZE = config.getint('server', 'max_request_size', fallback=8192)
SERVER_NAME = config.get('headers', 'server_name', fallback='MyPythonServer/1.0')

# Преобразуем относительный путь в абсолютный
if not os.path.isabs(WWW_DIR):
    WWW_DIR = os.path.join(os.path.dirname(__file__), WWW_DIR)

DEFAULT_PAGE = 'index.html'

def parse_request(request_text):
    lines = request_text.splitlines()
    if not lines:
        return None
    parts = lines[0].split()
    return parts[1] if len(parts) >= 3 else None

def get_file_path(uri):
    if uri.startswith('/'):
        uri = uri[1:]
```

```
    if not uri:
        uri = DEFAULT_PAGE
    return os.path.join(WWW_DIR, uri)

def build_headers(content_length, content_type='text/html', connection='close'):
    date_str = datetime.datetime.utcnow().strftime('%a, %d %b %Y %H:%M:%S GMT')
    return f"""Date: {date_str}
Server: {SERVER_NAME}
Content-Type: {content_type}
Content-Length: {content_length}
Connection: {connection}"""

def generate_error_page(status_code, status_text, description=""):
    html = f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>{status_code} {status_text}</title>
    <style>
        body {{ font-family: 'Segoe UI', sans-serif; background: #1a1a2e; color: #fff;
                display: flex; flex-direction: column; align-items: center;
                justify-content: center; height: 100vh; margin: 0; text-align: center; }}
        .code {{ font-size: 6em; color: #e94560; margin: 0; }}
        .text {{ font-size: 1.5em; color: #f5a623; }}
        .desc {{ color: #888; margin-top: 20px; }}
    </style>
</head>
<body>
    <p class="code">{status_code}</p>
    <p class="text">{status_text}</p>
    <p class="desc">{description}</p>
</body>
</html>"""
    return html

def handle_client(conn, addr):
    print(f"[+] Клиент {addr} подключён")
    try:
        data = conn.recv(MAX_REQUEST_SIZE)
        if not data:
            return

        msg = data.decode()
        uri = parse_request(msg)

        if uri is None:
            body = generate_error_page(400, "Bad Request")
            headers = build_headers(len(body.encode('utf-8')))
            resp = f"""HTTP/1.1 400 Bad Request
{headers}
```

```
{body}"""
        else:
            file_path = get_file_path(uri)

            if os.path.exists(file_path) and os.path.isfile(file_path):
                with open(file_path, 'r') as f:
                    content = f.read()
                headers = build_headers(len(content.encode('utf-8')))
                resp = f"""HTTP/1.1 200 OK
{headers}

{content}"""
            else:
                body = generate_error_page(404, "Not Found", f"Файл не найден: {uri}")
                headers = build_headers(len(body.encode('utf-8')))
                resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""

        conn.send(resp.encode())
    except Exception as e:
        print(f"[!] Ошибка: {e}")
    finally:
        conn.close()
        print(f"[-] Клиент {addr} отключён")

# === Сервер ===
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

try:
    sock.bind(('', PORT))
    print(f"Using port {PORT}")
except OSError:
    fallback_port = 8080 if PORT != 8080 else 8081
    sock.bind(('', fallback_port))
    print(f"Port {PORT} unavailable, using port {fallback_port}")

sock.listen(5)
print(f"Сервер {SERVER_NAME} запущен на порту {PORT}")
print("Нажмите Ctrl+C для остановки.")

try:
    while True:
        conn, addr = sock.accept()
        client_thread = threading.Thread(target=handle_client, args=(conn, addr))
        client_thread.daemon = True
        client_thread.start()
except KeyboardInterrupt:
    print("\n[!] Сервер остановлен")
```

```
finally:
    sock.close()
```

💡 : configparser INI- [section] key = value. get(), getint(), getboolean() fallback — , . .

4.9 — (access log) - . .

```
import socket
import os
import datetime
import threading
import configparser
import logging  # Стандартный модуль логирования

# === Настройка логирования ===
logging.basicConfig(
    filename='access.log',           # Файл журнала
    level=logging.INFO,              # Уровень важности
    format='%(asctime)s - %(message)s',  # Формат: дата — сообщение
    datefmt='%d.%m.%Y %H:%M:%S'     # Формат даты
)
# Также выводим логи в консоль
console = logging.StreamHandler()
console.setLevel(logging.INFO)
logging.getLogger('').addHandler(console)

logger = logging.getLogger('webserver')

# === Загрузка конфигурации ===
config = configparser.ConfigParser()
config.read('config.ini')

PORT = config.getint('server', 'port', fallback=8080)
WWW_DIR = config.get('server', 'www_dir', fallback='www')
MAX_REQUEST_SIZE = config.getint('server', 'max_request_size', fallback=8192)
SERVER_NAME = config.get('headers', 'server_name', fallback='MyPythonServer/1.0')

if not os.path.isabs(WWW_DIR):
    WWW_DIR = os.path.join(os.path.dirname(__file__), WWW_DIR)

DEFAULT_PAGE = 'index.html'

def parse_request(request_text):
    lines = request_text.splitlines()
    if not lines:
        return None, None
    parts = lines[0].split()
    if len(parts) >= 3:
        return parts[0], parts[1]  # Метод и URI
```

```
    return None, None

def get_file_path(uri):
    if uri.startswith('/'):
        uri = uri[1:]
    if not uri:
        uri = DEFAULT_PAGE
    return os.path.join(WWW_DIR, uri)

def build_headers(content_length, content_type='text/html', connection='close'):
    date_str = datetime.datetime.utcnow().strftime('%a, %d %b %Y %H:%M:%S GMT')
    return f"""Date: {date_str}
Server: {SERVER_NAME}
Content-Type: {content_type}
Content-Length: {content_length}
Connection: {connection}"""

def generate_error_page(status_code, status_text, description=""):
    html = f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>{status_code} {status_text}</title>
    <style>
        body {{ font-family: 'Segoe UI', sans-serif; background: #1a1a2e; color: #fff;
                display: flex; flex-direction: column; align-items: center;
                justify-content: center; height: 100vh; margin: 0; text-align: center; }}
        .code {{ font-size: 6em; color: #e94560; margin: 0; }}
        .text {{ font-size: 1.5em; color: #f5a623; }}
        .desc {{ color: #888; margin-top: 20px; }}
    </style>
</head>
<body>
    <p class="code">{status_code}</p>
    <p class="text">{status_text}</p>
    <p class="desc">{description}</p>
</body>
</html>"""
    return html

def handle_client(conn, addr):
    client_ip = addr[0]
    print(f"[+] Клиент {addr} подключён")

    try:
        data = conn.recv(MAX_REQUEST_SIZE)
        if not data:
            return

        msg = data.decode()
        method, uri = parse_request(msg)
```

```
        if uri is None:
            body = generate_error_page(400, "Bad Request")
            headers = build_headers(len(body.encode('utf-8')))
            resp = f"""HTTP/1.1 400 Bad Request
{headers}

{body}"""
            status_code = 400
        else:
            file_path = get_file_path(uri)

            if os.path.exists(file_path) and os.path.isfile(file_path):
                with open(file_path, 'r') as f:
                    content = f.read()
                headers = build_headers(len(content.encode('utf-8')))
                resp = f"""HTTP/1.1 200 OK
{headers}

{content}"""
                status_code = 200
            else:
                body = generate_error_page(404, "Not Found", f"Файл не найден: {uri}")
                headers = build_headers(len(body.encode('utf-8')))
                resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""
                status_code = 404

        conn.send(resp.encode())

        # Записываем в журнал: IP, метод, URI, код ответа
        logger.info(f"{client_ip} {method} {uri} {status_code}")

    except Exception as e:
        print(f"[!] Ошибка: {e}")
        logger.error(f"{client_ip} ERROR: {e}")
    finally:
        conn.close()
        print(f"[-] Клиент {addr} отключён")

# === Сервер ===
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

try:
    sock.bind(('', PORT))
    print(f"Using port {PORT}")
except OSError:
    sock.bind(('', 8080))
```

```
    print(f"Port {PORT} unavailable, using port 8080")

sock.listen(5)
print(f"Сервер {SERVER_NAME} запущен. Журнал: access.log")
print("Нажмите Ctrl+C для остановки.")

try:
    while True:
        conn, addr = sock.accept()
        client_thread = threading.Thread(target=handle_client, args=(conn, addr))
        client_thread.daemon = True
        client_thread.start()
except KeyboardInterrupt:
    print("\n[!] Сервер остановлен")
finally:
    sock.close()
```

:

```
# В реальном времени (обновляется по мере поступления запросов)
tail -f ~/pywebserver/access.log

# Полное содержимое
cat ~/pywebserver/access.log

# Фильтрация по коду ответа
grep " 404 " ~/pywebserver/access.log
```

📚 **:** `logging` — Python. , (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`), (handlers).

4.10 — (403 Forbidden) : (HTML, CSS, JS,). — 403.

```
# Добавьте в начало скрипта:
ALLOWED_EXTENSIONS = {'.html', '.htm', '.css', '.js', '.png', '.jpg', '.jpeg', '.gif', '.ico', '.svg'}

# Функция проверки расширения
def is_allowed_file(filename):
    """Проверяет, разрешено ли отдавать файл с данным расширением."""
    ext = os.path.splitext(filename)[1].lower()
    return ext in ALLOWED_EXTENSIONS

# В функции handle_client замените блок проверки файла на:
file_path = get_file_path(uri)
file_ext = os.path.splitext(file_path)[1].lower()

if not is_allowed_file(os.path.basename(file_path)) and os.path.basename(file_path) != '':
    body = generate_error_page(403, "Forbidden", f"Доступ к файлу запрещён: {uri}")
    headers = build_headers(len(body.encode('utf-8')))
    resp = f"""HTTP/1.1 403 Forbidden
```

```
{headers}

{body}"""
    status_code = 403
elif os.path.exists(file_path) and os.path.isfile(file_path):
    # ... существующий код для 200 OK
```

💡 : os.path.splitext() (имя_без_расширения, расширение). , os.path.splitext('style.css') → ('style', '.css'). .

4.11 — ( )

('rb') MIME- .

```
import mimetypes  # Модуль для определения MIME-типов по расширению

# Словарь расширений -> MIME-типов (дополнение к системным)
EXTRA_MIME_TYPES = {
    '.ico': 'image/x-icon',
    '.svg': 'image/svg+xml',
}

def get_content_type(file_path):
    """Определяет MIME-тип файла по его расширению."""
    ext = os.path.splitext(file_path)[1].lower()
    if ext in EXTRA_MIME_TYPES:
        return EXTRA_MIME_TYPES[ext]
    mime_type, _ = mimetypes.guess_type(file_path)
    return mime_type or 'application/octet-stream'

# В handle_client замените чтение файла на:
if os.path.exists(file_path) and os.path.isfile(file_path):
    content_type = get_content_type(file_path)

    # Открываем файл в бинарном режиме
    with open(file_path, 'rb') as f:
        content = f.read()  # content теперь bytes, не str

    headers = build_headers(len(content), content_type)
    # Формируем ответ: статусная строка + заголовки (str), затем пустая строка, затем content (bytes)
    resp_headers = f"""HTTP/1.1 200 OK
{headers}

"""
    # Объединяем заголовки (str→bytes) и тело (bytes)
    resp = resp_headers.encode('utf-8') + content

    conn.send(resp)  # resp уже bytes, не нужен .encode()
    status_code = 200
```

:

```
# Скачиваем тестовое изображение
curl -o ~/pywebserver/www/test.png https://via.placeholder.com/150

# Проверяем в браузере: http://localhost:8080/test.png
```

⚠️ : . 'rb' (read binary), bytes resp_headers.encode() + content. ('r') .

4.12 — HTTP/1.1 **persistent connections** (keep-alive): TCP- . .

```
def handle_client(conn, addr):
    client_ip = addr[0]
    print(f"[+] Клиент {addr} подключён")

    keep_alive = True
    request_count = 0

    try:
        while keep_alive:
            # Устанавливаем таймаут на чтение для keep-alive
            conn.settimeout(5)  # 5 секунд ожидания следующего запроса

            try:
                data = conn.recv(MAX_REQUEST_SIZE)
            except socket.timeout:
                # Таймаут — закрываем соединение
                break

            if not data:
                break

            request_count += 1
            msg = data.decode()
            method, uri = parse_request(msg)

            # Определяем, хочет ли клиент keep-alive
            connection_header = 'close'
            if 'Connection: keep-alive' in msg:
                connection_header = 'keep-alive'

            if uri is None:
                body = generate_error_page(400, "Bad Request")
                headers = build_headers(len(body.encode('utf-8')), connection=connection_header)
                resp = f"""HTTP/1.1 400 Bad Request
{headers}

{body}"""
```

```
                status_code = 400
            else:
                file_path = get_file_path(uri)

                if not is_allowed_file(os.path.basename(file_path)):
                    body = generate_error_page(403, "Forbidden", f"Доступ запрещён: {uri}")
                    headers = build_headers(len(body.encode('utf-8')), connection=connection_header)
                    resp = f"""HTTP/1.1 403 Forbidden
{headers}

{body}"""
                    status_code = 403
                elif os.path.exists(file_path) and os.path.isfile(file_path):
                    content_type = get_content_type(file_path)

                    with open(file_path, 'rb') as f:
                        content = f.read()

                    headers = build_headers(len(content), content_type, connection_header)
                    resp_headers = f"""HTTP/1.1 200 OK
{headers}

"""
                    resp = resp_headers.encode('utf-8') + content
                    conn.send(resp)
                    status_code = 200
                else:
                    body = generate_error_page(404, "Not Found", f"Файл не найден: {uri}")
                    headers = build_headers(len(body.encode('utf-8')), connection=connection_header)
                    resp = f"""HTTP/1.1 404 Not Found
{headers}

{body}"""
                    status_code = 404

            if isinstance(resp, str):
                conn.send(resp.encode())

            logger.info(f"{client_ip} {method} {uri} {status_code}")

            # Если клиент запросил close — завершаем цикл
            if connection_header == 'close':
                keep_alive = False

    except Exception as e:
        print(f"[!] Ошибка: {e}")
    finally:
        conn.close()
        print(f"[-] Клиент {addr} отключён (обработано запросов: {request_count})")
```

💡 : conn.settimeout(5) . 5 , socket.timeout, . .

Permission denied	bind	< 1024	root	8080	sudo
Address already in use				SO_REUSEADDR	60
				\n\n	
			Content-Type	charset=utf-8: text/html;	
				charset=utf-8	
UnicodeDecodeError			,	'rb'	
				threading.Thread	

```
# Проверка, какой процесс занимает порт 8080
sudo lsof -i :8080

# Или
sudo ss -tlnp | grep 8080

# Проверка соединения вручную через telnet
telnet localhost 8080
# Затем введите:
# GET / HTTP/1.1
# Host: localhost
# (пустая строка)

# Детальная информация о HTTP-запросе/ответе
curl -v http://localhost:8080/1.html

# Проверка заголовков ответа (без тела)
curl -I http://localhost:8080/1.html

# Мониторинг файла логов в реальном времени
tail -f ~/pywebserver/access.log

# Проверка MIME-типа файла
file --mime-type ~/pywebserver/www/1.html
```

```
~/pywebserver/
├── server.py          # Основной файл сервера
├── config.ini         # Файл конфигурации
```

```
├── access.log          # Журнал доступа (создаётся автоматически)
└── www/                # Корневая директория веб-сайта
    ├── index.html      # Главная страница
    ├── 1.html          # Тестовая страница 1
    ├── 2.html          # Тестовая страница 2
    └── test.png        # Тестовое изображение (опционально)
```

✅ **:** HTTP- Python , HTTP- , (400, 403, 404), , , , .

!

- HTTP- Python .

```
%%{init: {'theme': 'dark'}}%%
graph LR
    A["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f427.png}}}}}}$§ Linux Mint VM<br/>192.168.56.101"] --> B["§$
    ↪ \vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f310.png}}}}}}$§ site1.local"]
    A --> C["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f310.png}}}}}}$§ site2.local"]
    A --> D["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f510.png}}}}}}$§ SSH порт 22/2222"]
    A --> E["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f512.png}}}}}}$§ HTTPS 443"]
    A --> F["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f6e1-fe0f.png}}}}}}$§ UFW + Fail2Ban"]
    A --> Py["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f40d.png}}}}}}$§ Python HTTP-сервер<br/>порт 8080"]
    B --> G["§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 1f5a5-fe0f.png}}}}}}$§ Windows Host"]
    C --> G
    D --> G
    E --> G
    Py --> G

    style A fill:#87CF3E,stroke:#6aa82f,color:#000
    style G fill:#0078D4,stroke:#005A9E,color:#fff
    style E fill:#1a3d1a,stroke:#44ff44,color:#fff
    style F fill:#3d1f1f,stroke:#ff4444,color:#fff
    style Py fill:#3776AB,stroke:#2d5f8a,color:#fff
```

« »

Linux Mint 22 • Oracle VirtualBox 7.x • Nginx 1.24+ • Python 3.10+

04 : github.com/fa-python-network/6_Web_server

: 2026 .